

Apunte Física Computacional II

Integrantes: Joaquín Parra
Ignacio Falcón
Alvaro Osses
Benjamin Fuentes

Resumen

El siguiente documento presenta un apunte para el curso de Física Computacional II, basado en las notas de clase y presentaciones del ramo dictado por Dr. Roberto Navarro durante el segundo semestre de los años 2024 y 2025, y las notas de clase redactadas por Pedro Contreras.

Índice de Contenidos

1. Cálculo Numérico	1
1.1. Errores Computacionales	1
1.2. Derivadas numéricas	2
1.2.1. Otros métodos de derivación numérica	4
1.2.2. Derivadas para datos discretos	4
2. Breve introducción a Python	6
3. Introducción a la Búsqueda de Ceros	7
4. Interpolación y ajustes	8
4.1. Interpolación de Lagrange	8
4.2. Matriz de Vandermonde	9

Índice de Figuras

1. Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h	3
2. Conjunto de puntos interpolados por Lagrange y función original.	9

Índice de Códigos

1. Suma de dos decimales en Python	1
2. Output	1

1. Cálculo Numérico

1.1. Errores Computacionales

Si usted ha intentado realizar una suma de dos números decimales mediante algún lenguaje de programación, podrá haber notado que el resultado no es exactamente correcto

Código 1: Suma de dos decimales en Python

```
1 a = 0.1
2 b = 0.2
3 c = 0.3
4
5 print(f"{a + b == c}")
6 print(f"{a + b}")
```

Cuyo output será

Código 2: Output

```
1 False
2 0.30000000000000004
```

Este error se debe a la manera en que las computadoras deben representar números. En general, los humanos representamos los números en base 10, por ejemplo, escribimos el 325 como

$$325 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 = [325]_{10}$$

Pero los computadores utilizan bits (0 y 1), por lo que estos representan números en base 2, es decir:

$$325 = 2^8 + 2^6 + 2^2 + 2^0 = [101000101]_2$$

Ahora, para representar número reales en el sistema binario, se deben incluir exponentes negativos a la suma de exponentes. En la representación binaria, los números se separan por un exponente y una parte fraccionaria (mantisa) las cuales son separadas por un punto (.). Por ejemplo,

$$\begin{aligned}[2.145]_{10} &= 2 \cdot 10^0 + \frac{1}{8} \\ &= 2^1 + 2^{-3} \\ &= [10.001]_2 \\ &= [1.0001]_2 \cdot 2^1\end{aligned}$$

Este ejemplo en particular funciona bien porque la mantisa del número decimal resulta ser un exponente de 2, veamos que ocurre para un número más complejo:

$$\begin{aligned}
[0.1]_{10} &= [1]_2/[1010]_2 \\
&= [0.00011]_2 \\
&= [0.00011001100110011 \dots] \\
&= 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots \\
&= 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots)
\end{aligned}$$

Obtenemos una serie infinita de potencias de dos. Como en la practica los computadores tienen memoria finita es necesario truncar la serie. En general, la mantisa recibe 52 bits, por lo que la serie se trunca hasta el termino 2^{-52} . en efecto:

$$[0.1]_{10} = 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots + 2^{-52})$$

Este número no será *exactamente* igual a 0.1, si no que será igual a $0.1 \cdot (1 + \epsilon)$. Este será el caso para la gran mayoría de los calculos hechos computacionalmente, el resultado final tendrá una pequeña divergencia con respecto al valor real, es decir

$$\bar{a} = a \cdot (1 + \epsilon)$$

donde $\bar{a}$ y a son los valores calculado y real, respectivamente. La diferencia entre el número deseado y el resultante se denomina **error computacional**. Dado que las computadores no pueden físicamente lidiar con cantidades infinitesimalmente pequeñas, muchos de los métodos de cálculo adaptados a la programación que veremos a lo largo del apunte serán basados en aproximaciones, por esta razón es importante aprender a identificar fuentes de errores numericos y sus ordenes de magnitud.

1.2. Derivadas numéricas

Sabemos del calculo diferencial que la derivada de una función arbitraria f evaluada en un punto arbitrario x_0 es dada por

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h} \quad (1)$$

pero ya sabemos que necesitamos que h sea un número finito, por lo que estamos obligados a seguir anaizando esta definición. De la serie de taylor de f evaluada en un punto $a + h$, $h > 0$, tenemos que:

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(a)h^3 + \dots \quad (2)$$

Podemos reemplazar los terminos de orden superior por un $f(\xi)h^2$, $a \leq \xi \leq a + h$ (Teorema del valor medio), luego

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (3)$$

$$f'(a) = \frac{f(a + h) - f(a)}{h} - \frac{1}{2!}f''(\xi)h \quad (4)$$

Aquí, hemos obtenido una expresión finita para la derivada de una función evaluada en un punto, denominada *derivada adelantada*. Ahora, si buscamos el error de la ecuación (4) vemos que

$$Error = \left| f'(a) - \frac{f(a + h) - f(a)}{h} \right| = \frac{1}{2!} |f''(\xi)| h \quad (5)$$

en notación $\mathcal{O}$ de Landau,

$$Error = \frac{1}{2!} |f''(\xi)| h = \mathcal{O}(h) \quad (6)$$

Esto quiere decir que el error de truncamiento de la derivada adelantada será del orden de h , como se puede observar en la figura 1, el error de la derivada decrece linealmente con el orden de magnitud de h hasta aproximadamente $h \sim 10^{-8}$, pasado ese punto h se vuelve demasiado pequeño para la computadora y predomina el error computacional. Por esta razón, a la hora de implementar este, o cualquiera de los métodos expuestos en este apunte, es conveniente utilizar un $h \sim 10^{-5}$.

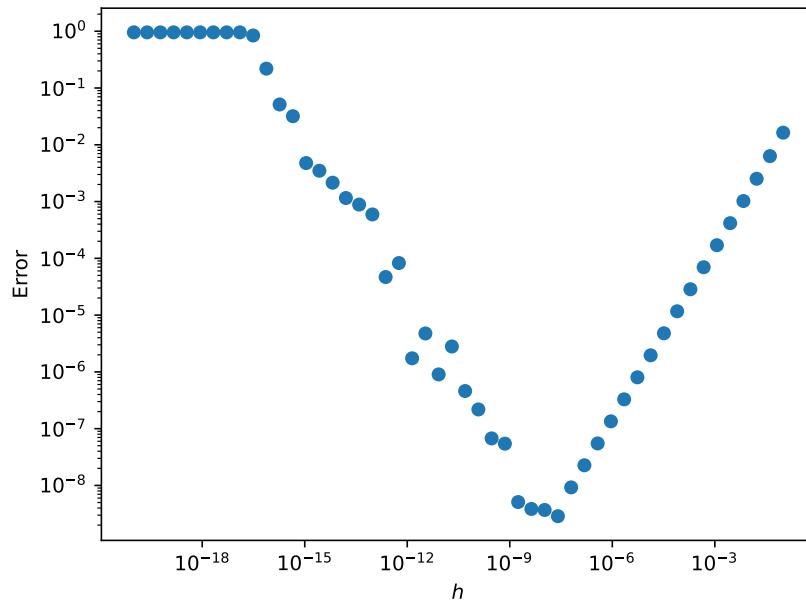


Figura 1: Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h

1.2.1. Otros métodos de derivación numérica

Si bien la derivada adelantada es útil debido a lo simple de su implementación, muchas veces queremos utilizar métodos cuyo error sea de un mayor orden de h , es decir, menor.

Similarmente a como obtuvimos la expresión (4), podemos obtener una derivada **retrasada**, en efecto, consideramos la serie de taylor de $f(a - h)$,

$$f(a - h) = f(a) - f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (7)$$

$$f'(a) = \frac{f(a) - f(a - h)}{h} + \frac{1}{2!}f''(\xi)h \quad (8)$$

Si consideramos las series de taylor de $f(a + h)$ y $f(a - h)$ expandidas hasta el tercer orden,

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(\xi)h^3 \quad (9)$$

$$f(a - h) = f(a) - f'(a)h + \frac{1}{2!}f''(a)h^2 - \frac{1}{3!}f'''(\xi)h^3 \quad (10)$$

Ahora si restamos obtenemos,

$$f(a + h) - f(a - h) = 2f'(a)h + \frac{2}{3!}f'''(\xi)h^3 \quad (11)$$

$$2f'(a)h = f(a + h) - f(a - h) - \frac{2}{3!}f'''(\xi)h^3 \quad (12)$$

$$f'(a) = \frac{f(a + h) - f(a - h)}{2h} - \frac{1}{6}f'''(\xi)h^3 \quad (13)$$

Así, obtenemos una expresión para la **derivada centrada**, con error del orden de h^3 .

Podemos obtener muchos otros métodos de derivación numérica por medios similares, en general el proceso consiste en jugar con series de taylor hasta poder despejar el termino $f'(a)$.

1.2.2. Derivadas para datos discretos

En general, uno no suele tratar con funciones definidas analíticamente, si no que tendremos conjuntos de datos relacionados a alguna magnitud física, si quisieramos eventualmente encontrar la derivada de la "función" que definen estos datos, debemos aprender a tratar con puntos no necesariamente equidistantes.

Similarmente a caso anterior, dado un conjunto de puntos x_k y una función continua y diferenciable tal que $f(x_k) = f_k$, definimos la distancia entre dos puntos como $h_k = x_k - x_{k-1}$. Podemos obtener una primera aproximación simplemente tomando la pendiente entre dos puntos, esto es

$$\frac{df_k}{dx_k} \approx \frac{f_k - f_{k-1}}{x_k - x_{k-1}} \quad (14)$$

Esto es analogo a la derivada atrasada encontrada en la sección anterior, y se puede demostrar

que tendrá un error del orden de h_k . Notar además que si imponemos que los nodos estén igualmente espaciados, es decir, $x_k - x_{k-1} = h = cte.$, la ecuación (14) recupera la ecuación (8).

Ahora, estamos incentivados a encontrar un método de derivación discreta con error de menor orden, para ello consideramos la serie de Taylor de f_{k+1} y f_{k-1}

$$f_{k+1} = f_k + \frac{df_k}{dx}h_{k+1} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_{k+1}^2 + \frac{1}{3!}\frac{d^3f(\xi_1)}{dx^3}h_{k+1}^3 \quad (15)$$

$$f_{k-1} = f_k - \frac{df_k}{dx}h_k + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_k^2 - \frac{1}{3!}\frac{d^3f(\xi_2)}{dx^3}h_k^3 \quad (16)$$

con $x_{k-1} \leq \xi_2 \leq x_k \leq \xi_1 \leq x_{k+1}$. Despejando $\frac{df_k}{dx}$ en ambas expresiones obtenemos,

$$\frac{df_k}{dx}h_{k+1} = f_{k+1} - f_k - \frac{1}{2!}\frac{d^2f_k}{dx^2}h_{k+1}^2 - \frac{d^3f(\xi_1)}{dx^3}h_{k+1}^3 \quad (17)$$

$$\frac{df_k}{dx}h_k = f_k - f_{k-1} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_k^2 + \frac{d^3f(\xi_2)}{dx^3}h_k^3 \quad (18)$$

Multiplicando (17) y (18) por h_k/h_{k+1} y h_{k+1}/h_k , respectivamente,

$$\frac{df_k}{dx}h_k = \frac{(f_{k+1} - f_k)h_k}{h_{k+1}} - \frac{1}{2!}\frac{d^2f_k}{dx^2}h_kh_{k+1} - \frac{d^3f(\xi_1)}{dx^3}h_{k+1}^2h_k \quad (19)$$

$$\frac{df_k}{dx}h_{k+1} = \frac{(f_k - f_{k-1})h_{k+1}}{h_k} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_kh_{k+1} + \frac{d^3f(\xi_2)}{dx^3}h_k^2h_{k+1} \quad (20)$$

Sumando,

$$[h_{k+1} + h_k]\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})h_{k+1}}{h_k} + \frac{1}{3!}[f'''(\xi_2)h_k - f'''(\xi_1)h_{k+1}]h_kh_{k+1} \quad (21)$$

Finalmente,

$$\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \frac{1}{3!} \left[\frac{f'''(\xi_2)h_k - f'''(\xi_1)h_{k+1}}{h_{k+1} + h_k} \right] h_kh_{k+1} \quad (22)$$

Ahora, por el teorema del valor medio, podemos reemplazar el último termino entre corchetes por una función evaluada en un punto ξ , con $\xi_2 \leq \xi \leq \xi_1$, en efecto,

$$\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \frac{1}{3!} \frac{d^3f_k(\xi)}{dx^3} h_kh_{k+1} \quad (23)$$

$$= \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \mathcal{O}(h_kh_{k+1}) \quad (24)$$

Así, logramos encontrar una expresión para la derivada de una función definida punto

a punto con nodos no necesariamente equidistantes, con error de orden cuadrático. Puede comprobar que si imponemos que los nodos sean equidistantes ($h_k = h_{k+1} = h$), se recupera la expresión (13) para la derivada centrada (**Demuestrelo!**). Más adelante estudiaremos métodos más avanzados para tratar con distribuciones de datos con interpolaciones y splines. Para terminar, cabe mencionar que la mayor desventaja de este método es que, al utilizar 3 puntos, aplicándola a una serie de puntos, la expresión numérica de la derivada tendrá dos puntos menos que la serie original. Por esta razón, en la práctica se implementa una derivada adelantada para los puntos iniciales de un set de datos, y una atrasada para los finales; la derivada centrada se utiliza solo para los puntos intermedios.

2. Breve introducción a Python

Hola

3. Introducción a la Búsqueda de Ceros

Usualmente, cuando trabajamos con ecuaciones, una de nuestras metas es encontrar el valor de cierta variable o también llamada incógnita, para esto, desarrollamos una serie de pasos o "algoritmo" que seguir para encontrar la respuesta. Un buen ejemplo de esto es considerar el caso de una ecuación cuadrática:

$$ax^2 + bx^2 + c = 0 \quad (25)$$

Es bien sabido que, dado los coeficientes a, b y c, existe una fórmula general que seguir, la conocida como ecuación cuadrática o *Ecuación de Bashkara*:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (26)$$

Al introducir los respectivos valores de los coeficientes, podemos encontrar las dos soluciones a este problema.

Sin embargo, no siempre se puede encontrar una ecuación para los valores de incógnitas en toda ecuación, un ejemplo a esto es el siguiente:

$$\sin x + x = 0 \quad (27)$$

A pesar de lo simple de nuestro problema (puesto que es la suma entre una función trigonométrica y la incógnita), ya no es posible encontrar una ecuación de forma analítica que exprese valores exactos de las soluciones para "x", así, la pregunta es ¿Qué podemos hacer en estos casos?

Para esto, se desarrollan los **Métodos para hallar ceros de una función**

meow

4. Interpolación y ajustes

Como mencionamos muy anteriormente, muchas veces no tendremos el lujo de lidiar con funciones definidas analíticamente en todos sus puntos, en lugar de eso, tendremos algún conjunto discreto de puntos $(x_i, f_i)_{i=0, \dots, N}$. En estos casos podríamos querer hacer una de dos cosas: interpolar, o bien, ajustar.

4.1. Interpolación de Lagrange

La interpolación consiste en encontrar una función que pase por todos y cada uno de los puntos dados. En estricto rigor estamos llenando el espacio entre medio de los puntos. Si le suena familiar, es porque esto es lo que hacen los modelos de lenguaje (Gemini, ChatGPT, Claude, etc); la diferencia es que estos lo hacen con sets de datos enormes. Ahora, dados un conjunto arbitrario de puntos $(x_i, f_i)_{i=0, \dots, N}$ siempre podemos encontrar un polinomio de grado menor o igual a N que interpole los puntos, es decir

$$f_i = p(x_i) \quad (28)$$

Podemos proponer una combinación lineal de polinomios de la forma

$$L(x) = \sum_{j=0}^N f_j \ell_j(x) \quad (29)$$

imponiendo la condición

$$\ell_j(x_i) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad (30)$$

con tal que se cumpla (28). Podemos encontrar que los polinomios $\ell_j(x)$, denominados **polinmios base de Lagrange**, son dados por:

$$\ell_j(x) = \left(\frac{x - x_0}{x_j - x_0} \right) \left(\frac{x - x_1}{x_j - x_1} \right) \dots \left(\frac{x - x_{j-1}}{x_j - x_{j-1}} \right) \left(\frac{x - x_{j+1}}{x_j - x_{j+1}} \right) \dots \left(\frac{x - x_N}{x_j - x_N} \right) \quad (31)$$

$$= \prod_{i=0, i \neq j}^N \left(\frac{x - x_i}{x_j - x_i} \right) \quad (32)$$

y forman (29), denominado **polinomio interpolador de Lagrange**.

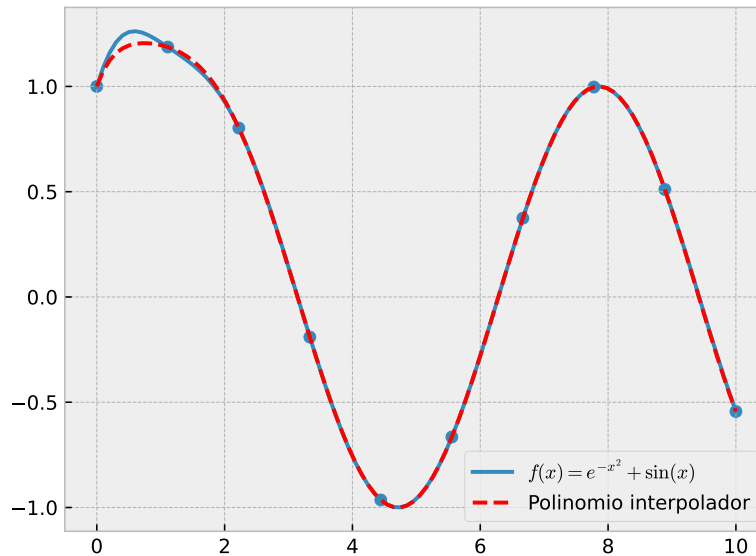


Figura 2: Conjunto de puntos interpolados por Lagrange y función original.

4.2. Matriz de Vandermonde

En la sección anterior, logramos interpolar un conjunto de puntos utilizando una combinación lineal de polinomios (vea (29)), por lo que es natural preguntarse si existe algún otro tipo de función interpoladora. La respuesta es que podemos interpolar con cualquier tipo de función que forme una **base del espacio de funciones**. Consideremos que queremos interpolar un conjunto de puntos (x_j, f_j) por medio de una función $p(x)$ tal que

$$f_j = p(x_j)$$

tenemos que

$$p(x) = \sum_{i=0}^N p_i \varphi_i(x) \quad (33)$$

$$= p_0 \varphi_0(x) + p_1 \varphi_1(x) + \cdots + p_N \varphi_N(x) \quad (34)$$

Así, tenemos el sistema de ecuaciones

$$f_j = p_0 \varphi_0(x_j) + p_1 \varphi_1(x_j) + \cdots + p_N \varphi_N(x_j) \quad (35)$$

donde queremos encontrar los p_j . Matricialmente tenemos que

$$\begin{bmatrix} \varphi_0(x_0) & \varphi_1(x_0) & \cdots & \varphi_N(x_0) \\ \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_N(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_0(x_N) & \varphi_1(x_N) & \cdots & \varphi_N(x_N) \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_N \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_N \end{bmatrix} \quad (36)$$

De esta manera, reducimos el problema a resolver el sistema encontrando la inversa de la matriz $\varphi_i(x_j)$. Finalmente, todo este proceso es una generalización de encontrar los coeficientes de una serie que aproxime una función, y es analogo a encontrar los coeficientes de una serie de Taylor. El tipo de serie por el cual estemos interpolando depende de como elijamos la función $\varphi_n(x)$, por ejemplo

- Serie de Fourier: $\varphi_n(x) = \exp^{in\pi x/L}$
- Serie de polinomios de Lagrange: $\varphi_n(x) = \ell_n(x)$
- Serie de Legendre: $\varphi_n(x) = P_n(x)$ (Polinomios de Legendre)